

<https://www.geeksforgeeks.org/program-for-priority-cpu-scheduling-set-1/>

```
// Java program for implementation of FCFS
// scheduling
import java.util.*;

class Process
{
    int pid; // Process ID
    int bt; // CPU Burst time required
    int priority; // Priority of this process
    Process(int pid, int bt, int priority)
    {
        this.pid = pid;
        this.bt = bt;
        this.priority = priority;
    }
    public int prior() {
        return priority;
    }
}

public class GFG
{
    // Function to find the waiting time for all
    // processes
    public void findWaitingTime(Process proc[], int n,
                                int wt[])
    {
        // waiting time for first process is 0
        wt[0] = 0;

        // calculating waiting time
        for (int i = 1; i < n ; i++)
            wt[i] = proc[i - 1].bt + wt[i - 1] ;
    }

    // Function to calculate turn around time
    public void findTurnAroundTime( Process proc[], int n,
                                    int wt[], int tat[])
    {
        // calculating turnaround time by adding
        // bt[i] + wt[i]
        for (int i = 0; i < n ; i++)
            tat[i] = proc[i].bt + wt[i];
    }

    // Function to calculate average time
    public void findavgTime(Process proc[], int n)
    {
        int wt[] = new int[n], tat[] = new int[n], total_wt = 0, total_tat = 0;
```

```

        // Function to find waiting time of all processes
        findWaitingTime(proc, n, wt);

        // Function to find turn around time for all processes
        findTurnAroundTime(proc, n, wt, tat);

        // Display processes along with all details
        System.out.print("\nProcesses    Burst time    Waiting time    Turn around
time\n");

        // Calculate total waiting time and total turn
        // around time
        for (int i = 0; i < n; i++)
        {
            total_wt = total_wt + wt[i];
            total_tat = total_tat + tat[i];
            System.out.print(" " + proc[i].pid + "\t\t" + proc[i].bt + "\t " +
wt[i] + "\t\t" + tat[i] + "\n");
        }

        System.out.print("\nAverage waiting time = "
            +(float)total_wt / (float)n);
        System.out.print("\nAverage turn around time = " + (float)total_tat /
(float)n);
    }

    public void priorityScheduling(Process proc[], int n)
    {

        // Sort processes by priority
        Arrays.sort(proc, new Comparator<Process>() {
            @Override
            public int compare(Process a, Process b) {
                return b.prior() - a.prior();
            }
        });
        System.out.print("Order in which processes gets executed \n");
        for (int i = 0 ; i < n; i++)
            System.out.print(proc[i].pid + " ");

        findavgTime(proc, n);
    }

    // Driver code
    public static void main(String[] args)
    {
        GFG ob=new GFG();
        int n = 3;
        Process proc[] = new Process[n];
        proc[0] = new Process(1, 10, 2);
        proc[1] = new Process(2, 5, 0);
        proc[2] = new Process(3, 8, 1);
        ob.priorityScheduling(proc, n);
    }
}

```

// This code is contributed by rahulpatil07109.

<https://www.geeksforgeeks.org/program-round-robin-scheduling-set-1/>

Round Robin Sample Program

// Java program for implementation of RR scheduling

```
public class GFG
{
    // Method to find the waiting time for all
    // processes
    static void findWaitingTime(int processes[], int n,
                                int bt[], int wt[], int quantum)
    {
        // Make a copy of burst times bt[] to store remaining
        // burst times.
        int rem_bt[] = new int[n];
        for (int i = 0 ; i < n ; i++)
            rem_bt[i] = bt[i];

        int t = 0; // Current time

        // Keep traversing processes in round robin manner
        // until all of them are not done.
        while(true)
        {
            boolean done = true;

            // Traverse all processes one by one repeatedly
            for (int i = 0 ; i < n; i++)
            {
                // If burst time of a process is greater than 0
                // then only need to process further
                if (rem_bt[i] > 0)
                {
                    done = false; // There is a pending process

                    if (rem_bt[i] > quantum)
                    {
                        // Increase the value of t i.e. shows
                        // how much time a process has been processed
                        t += quantum;

                        // Decrease the burst_time of current process
                        // by quantum
                        rem_bt[i] -= quantum;
                    }

                    // If burst time is smaller than or equal to
```

```

        // quantum. Last cycle for this process
        else
        {
            // Increase the value of t i.e. shows
            // how much time a process has been processed
            t = t + rem_bt[i];

            // Waiting time is current time minus time
            // used by this process
            wt[i] = t - bt[i];

            // As the process gets fully executed
            // make its remaining burst time = 0
            rem_bt[i] = 0;
        }
    }

    // If all processes are done
    if (done == true)
        break;
}

// Method to calculate turn around time
static void findTurnAroundTime(int processes[], int n,
                               int bt[], int wt[], int tat[])
{
    // calculating turnaround time by adding
    // bt[i] + wt[i]
    for (int i = 0; i < n ; i++)
        tat[i] = bt[i] + wt[i];
}

// Method to calculate average time
static void findavgTime(int processes[], int n, int bt[],
                        int quantum)
{
    int wt[] = new int[n], tat[] = new int[n];
    int total_wt = 0, total_tat = 0;

    // Function to find waiting time of all processes
    findWaitingTime(processes, n, bt, wt, quantum);

    // Function to find turn around time for all processes
    findTurnAroundTime(processes, n, bt, wt, tat);

    // Display processes along with all details
    System.out.println("Processes " + " Burst time " +
                       " Waiting time " + " Turn around time");

    // Calculate total waiting time and total turn
    // around time
    for (int i=0; i<n; i++)
    {

```

```

        total_wt = total_wt + wt[i];
        total_tat = total_tat + tat[i];
        System.out.println(" " + (i+1) + "\t\t" + bt[i] + "\t " +
                           wt[i] + "\t\t" + tat[i]);
    }

    System.out.println("Average waiting time = " +
                       (float)total_wt / (float)n);
    System.out.println("Average turn around time = " +
                       (float)total_tat / (float)n);
}

// Driver Method
public static void main(String[] args)
{
    // process id's
    int processes[] = { 1, 2, 3};
    int n = processes.length;

    // Burst time of all processes
    int burst_time[] = {10, 5, 8};

    // Time quantum
    int quantum = 2;
    findavgTime(processes, n, burst_time, quantum);
}
}

```